Optimization Log Copilot: Using Large Language Models to Analyze Mathematical Optimization Solver Logs

Jeff Ginn
EN.705.605 Introduction to Generative AI

May 3, 2026

Abstract

Modern optimization solvers for linear programming (LP) and mixed-integer programming (MIP) produce detailed logs describing solver progress, including presolve reductions, cutting plane generation, branching decisions, heuristic activity, and bound progression. Interpreting these logs to diagnose performance bottlenecks requires significant domain expertise and manual effort, limiting accessibility for many practitioners. This paper presents the Optimization Log Copilot, a system that combines structured log parsing, heuristic-rule weak supervision, a multi-label PyTorch classifier, and retrieval-augmented generation to produce automated diagnostic summaries of solver runs. The system was evaluated on 188 solver runs collected from MIPLIB 2017 benchmarks and Ramsey-type problems under two builds of the Rose solver, with an independent 30-run hand-reviewed gold subset providing ground truth. The trained classifier achieved macro-F1 of 0.874 on a held-out validation split and 0.903 on the gold subset, exceeding the proposed 0.85 target and improving on its weak-supervision noise floor on one labeled category. The retrieval-augmented diagnostic generator, using `Llama-3.2-1B-Instruct` with label-conditioned retrieval, produced coherent paragraph-length diagnoses on bottleneck and healthy runs. A direct test of whether the system's diagnostic output can recommend the better of the two solver configurations failed to find a signal: a paired Wilcoxon test on per-problem solve times did not reject equality of distributions ($p = 0.43$), and neither a rule-based nor an LLM-based picker exceeded the majority-class baseline. We report this as a negative result and discuss why the diagnostic taxonomy does not transfer cleanly to the configuration-selection task.

1 Introduction

Modern optimization solvers for linear programming (LP) and mixed-integer linear programming (MILP) produce logs that describe solver progress in detail, including presolve activity, cut generation, heuristic behavior, branching decisions, node exploration, optimality gap progression, and termination status. While these logs contain valuable diagnostic information, they are difficult to interpret without deep expertise in optimization algorithms. As a result, users often struggle to understand why a solver run is slow, fails to converge, or exhibits unstable behavior. Diagnosing these issues typically requires manual inspection, which limits accessibility and productivity.

The difficulty of solver log analysis stems from several factors. First, solver logs are semi-structured text that mix numerical output with free-form status messages. Second, the logs can be extremely long, sometimes containing thousands of lines for a single solve. Third, the terminology and metrics are domain-specific: concepts such as LP relaxation gaps, simplex pivots, Gomory cuts, and branching pseudocosts require familiarity with mathematical optimization theory. Finally, the patterns that indicate performance issues are often subtle and context-dependent. For example, a

large root LP gap alone may or may not indicate a weak formulation, depending on the problem structure and subsequent bound improvement.

Recent advances in large language models (LLMs) have demonstrated strong capabilities in processing semi-structured text, including system log analysis for incident detection [1], anomaly detection in software infrastructure [2], and reasoning over structured data. However, these techniques have been applied primarily to IT system logs and software engineering contexts, not to mathematical optimization solver logs.

This project explores whether LLMs can serve as an "Optimization Log Copilot" that automatically analyzes solver logs and provides diagnostic insights about solver performance. The system aims to answer questions such as: Why is the solver exploring a large number of nodes? Is the root LP relaxation weak? Are cutting planes improving the bound? What structural issues might exist in the optimization model?

The proposed approach combines structured log parsing with transformer-based sequence understanding and conditional text generation. The system is designed as a post-processing advisory layer that operates independently of solver internals. The ML problems addressed include multi-label classification for bottleneck detection and sequence-to-sequence generation for diagnostic explanation.

The remainder of this paper is organized as follows. Section 2 surveys related work in LLM-based log analysis, deep learning for anomaly detection, and machine learning for combinatorial optimization. Section 3 defines the research problem and hypotheses. Section 4 describes the proposed method, including the system architecture, dataset, and training approach. Section 5 outlines the planned experimental evaluation. Section 6 presents results. Section 7 discusses conclusions and future directions.

2 Related Work

Several research areas inform this project: LLM-based log analysis, deep learning for anomaly detection in log data, and machine learning for combinatorial optimization.

2.1 LLM-Based Log Analysis

Akhtar et al. [1] survey how large language models are being applied to event log analysis, particularly in system reliability and software engineering contexts. They identify that traditional approaches to log analysis require heavy feature engineering, depend on rigid log parsing, and struggle with unstructured or evolving log formats. The survey categorizes LLM-based approaches into four groups: fine-tuning, in-context learning, retrieval-augmented generation (RAG), and hybrid symbolic–LLM systems. A key finding is that RAG is strongly preferred over fine-tuning for log analysis because fine-tuning LLMs on logs is expensive and brittle, while RAG performs well with smaller datasets and reduces hallucination risk. The survey also notes that log analysis evaluation often lacks ground truth and that human judgment remains necessary, suggesting that an LLM-based copilot should supplement rather than replace human reasoning.

2.2 Deep Learning for Anomaly Detection in Log Data

Landauer et al. [2] survey deep learning methods for log anomaly detection, covering system failures, incidents, intrusions, and operational breakdowns. The methods reviewed include sequence-based models (LSTMs, GRUs, and Transformer architectures), reconstruction-based models (autoencoders and variational autoencoders), and graph-based methods. Key findings include that

transformer models outperform LSTMs in modeling long-range dependencies, and that deep models generally outperform classical statistical anomaly detectors, with reported F1 scores ranging from 0.90 to 0.98 for deep LSTM-based models and often exceeding 0.95 for transformer-based approaches. Common evaluation metrics include precision, recall, F1 score, and ROC-AUC. However, these systems have been applied primarily to software infrastructure logs rather than optimization solver logs, leaving the domain of solver diagnostics largely unexplored.

2.3 Machine Learning for Combinatorial Optimization

Bengio et al. [3] survey how machine learning can assist combinatorial optimization solvers by learning decision policies. They categorize ML usage into learning to configure (parameter tuning, solver selection), learning to guide search (branching policies, cut selection, node selection), learning surrogate models (predicting solver runtime and node counts), and reinforcement learning for search policies. The survey reports that learned branching strategies can yield 10–30% runtime reductions over handcrafted heuristics, and that ML-based solver configuration can reduce runtime significantly. For this project, the survey underscores that ML-driven solver diagnostics and decision support are important research directions, particularly for recommending parameter settings such as cut aggressiveness, heuristic configurations, and presolve adjustments based on log analysis.

Kotary et al. [4] survey end-to-end constrained optimization learning, providing additional context on how ML models can be integrated with optimization solvers. Yazdani et al. [5] demonstrate how language models can be used to strengthen integer programs through evolution-guided cut generation, illustrating a direct connection between LLMs and optimization solver improvement.

2.4 Gap in the Literature

While LLM-based log analysis has been studied extensively for IT infrastructure and software systems, and machine learning has been applied to various aspects of optimization solver control, using LLMs to analyze and diagnose optimization solver logs remains largely unexplored. Optimization researchers typically analyze solver logs manually through ad hoc inspection of branch-and-bound traces, cut effectiveness metrics, and solver benchmarking output. Automated interpretation tools for solver logs are rare. This project addresses that gap.

3 Research Problem and Hypotheses

3.1 Problem Description

Solver logs are semi-structured sequences containing solver events, numerical metrics, and domain-specific terminology. Modern solvers internally adapt their behavior based on problem structure adjusting branching strategies, selecting cuts, and configuring heuristics, but this information is encoded in log output that is not easily accessible to users. The research problem addressed in this project is:

> *Given a raw solver log from a LP/MIP solver, can a machine learning system generate a structured diagnostic summary and identify likely performance bottlenecks?*

Typical performance issues that the system should detect include:

- Weak LP relaxation: Large initial optimality gap at the root node, indicating the LP relaxation provides a poor bound.

- Excessive branching: Exploding node counts during branch-and-bound search, often caused by weak variable selection or symmetry.
- Ineffective cut generation: Many cutting plane rounds with little bound improvement.
- Degeneracy: Slow LP pivots and numerical instability during the simplex method.
- Insufficient presolve: Minimal problem reduction during the presolve phase.

3.2 Research Hypotheses

The project investigates three research questions, formalized as testable hypotheses:

- H1 (Summarization): An LLM-based system can generate accurate natural-language summaries of solver log behavior, as measured by ROUGE scores and human evaluation against expert-written summaries.
- H2 (Bottleneck Detection): Solver logs contain sufficient information to classify the cause of poor solver performance, and a trained classifier can achieve F1 scores comparable to those reported in related log anomaly detection work (>0.85).
- H3 (Recommendations): The system can produce actionable solver configuration recommendations that correlate with measurable improvements in solver performance metrics such as solve time and node count.

4 Method

The Optimization Log Copilot combines structured log parsing, transformer-based sequence understanding, and generative language models to produce both structured predictions and natural-language explanations. The system does not modify solver internals.

4.1 System Architecture

The system pipeline consists of four stages, shown in Figure 1.

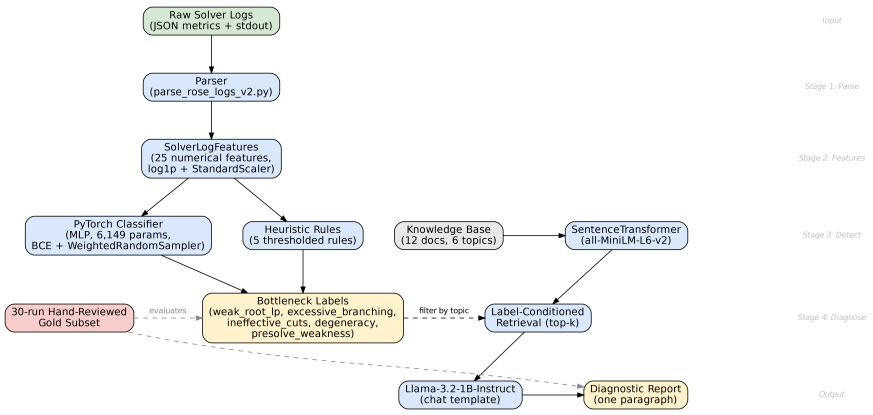

Figure 1: Optimization Log Copilot system architecture. Raw Rose solver logs (JSON metrics + stdout) are parsed into a `SolverLogFeatures` representation. Bottleneck detection runs along two parallel paths – heuristic threshold rules and a PyTorch MLP classifier – both producing a five-label multi-label vector. Diagnostic generation uses label-conditioned retrieval over a curated knowledge base (top-k entries filtered to the detected labels) fed to `Llama-3.2-1B-Instruct`. A 30-run hand-reviewed gold subset evaluates both the bottleneck labels (per-label F1, ROC-AUC) and the generated diagnostics (qualitative review).

1. Log Parsing: Each Rose solver run produces a directory of structured JSON files (`status.json`, `basic_metrics.json`, `problem.json`, `detailed_metrics.json`, `bounds.json`) plus a gzipped `stdout.txt`. A Python parser walks the directory, loads the JSON files, parses the stdout (with ANSI codes stripped) for additional features, computes derived ratios (presolve reduction percentages, dual-bound improvement, LP iterations per variable), and emits one CSV row per run with 25 numerical features plus identifying columns.

2. Feature Representation: Parsed runs are wrapped in a `SolverLogFeatures` dataclass that captures problem dimensions, solution status, timing, tree-search statistics, cut counts, bound trajectory, and the derived ratios. Count-valued features are log-transformed via $\log(1 + x)$ to handle the heavy-tailed distributions observed in the dataset (node counts span six orders of magnitude), and all features are then standardized via z-score normalization fit on the training split.

3. Bottleneck Detection: Two parallel paths produce a five-label multi-label vector. Heuristic threshold rules (Section 4.2) provide weak supervision; a PyTorch MLP trained on those rule-derived labels (Section 4.3) provides a learned counterpart that can in principle generalize beyond the rule-detected positives.

4. Diagnostic Generation: A 12-document curated knowledge base, embedded with the `all-MiniLM-L6-v2` sentence transformer, is retrieved against using a query built from the parsed run summary. Retrieval is filtered to the topics matching the detected bottleneck labels (or the `healthy_solve` topic when no labels fire). The retrieved context, the labels, and a one-line narrative summary of the run are then passed to `Llama-3.2-1B-Instruct` via its chat template, which produces a single-paragraph diagnostic report.

4.2 Dataset Construction

The dataset consists of 188 solver runs across 98 unique optimization problems, collected from two sources:

- MIPLIB 2017 benchmarks: 90 standard benchmark instances from the Mixed Integer Programming Library (`https://miplib.zib.de/`), each run twice under two different builds of the Rose solver, yielding 180 paired runs. The two configurations are referred to throughout the paper as `branching_heavy` and `heuristic_heavy` based on their observed behavior: across the 90 paired problems, `branching_heavy` (Rose 5.23.1-develop-4-g160c19936, staging branch) explored 44% more branch-and-bound nodes per problem at the median, while `heuristic_heavy` (Rose 5.23.1-release) found 13% more incumbents and was slightly faster overall. The two builds share an identical exposed control configuration (`cut_strategy`, `heuristic_strategy`, parallelism, gap tolerances, random seed), so the behavioral difference reflects internal heuristic-policy changes between the two builds rather than user-set parameters.

- Ramsey-type problems: A series of 8 combinatorial models representing Ramsey problems (e.g., $R(3, 4; 9)$), which exhibit a range of solver behaviors including symmetry, large search trees, infeasibility, and difficult relaxations.

Logs were generated using the Rose solver. The 90 paired MIPLIB runs are the primary basis for the configuration-comparison analysis (H3), while the full 188-run dataset is used for classifier training and diagnostic-generation evaluation. The runs cover a wide range of solver behaviors, including 154 optimal terminations, 21 feasible-but-suboptimal terminations (mostly timeouts), and 13 instances with no solution found; node counts span four to eight orders of magnitude depending on the problem.

Each training example is constructed as a triple:

$$(\text{log_text}, \text{diagnostic_labels}, \text{explanation}) \tag{1}$$

where log_text is the cleaned solver log, diagnostic_labels is a multi-hot vector of bottleneck categories, and explanation is a natural-language summary describing solver behavior and potential improvements. Labels are generated using heuristic rules derived from solver metrics. Each rule combines a primary signal with a guard so that easy cases (problems solved entirely in presolve, tiny problems) do not trigger spurious labels. The rules are: (i) `weak_root_lp` fires when the final relative gap is at least 5%, or no solution was found and the problem was not solved in presolve; (ii) `excessive_branching` fires when the node count per variable is at least 100 and the absolute node count is at least 1,000; (iii) `ineffective_cuts` fires when at least 100 cuts were generated, the dual bound moved less than 1% from root LP to final, and the problem was not solved in presolve; (iv) `degeneracy` fires when LP iterations per variable is at least 1.0 and the problem has at least 10 variables; (v) `presolve_weakness` fires when both variable and constraint reductions are below 1% and the problem was not solved in presolve.

To validate the heuristic supervision, a stratified subset of 30 solver runs spanning both configurations and the full problem-size range (16 to 345,856 variables) was hand-reviewed against the underlying metric evidence. The expert labels were compared to the heuristic rules, yielding macro-F1 of 0.900 and micro-F1 of 0.929. Recall was 1.0 across all five labels, indicating the rules do not miss positives in the reviewed sample. Precision losses were concentrated on two labels, `degeneracy` (0.71) and `presolve_weakness` (0.50), with all false positives occurring on small instances (fewer than 50 variables) where the rule conditions fired but the underlying phenomenon

was not meaningfully present (e.g., presolve has little to reduce on a 30-variable model). This residual noise pattern is documented and treated as a property of the supervision rather than a tuning target, since modifying the rules using the gold subset would compromise its independence as the evaluation instrument for the downstream classifier and diagnostic-generation experiments.

4.3 Bottleneck Classification Model

The classification component is formulated as a multi-label classification problem. Given a solver log, the model predicts a probability for each bottleneck category. The label set includes:

Label	**Indicator**
Weak root LP	Large initial optimality gap
Branching difficulty	High node count
Degeneracy	Slow LP pivots
Ineffective cuts	Many rounds, little bound improvement
Presolve weakness	Minimal variable/constraint reduction

The current implementation uses a feed-forward neural network classifier operating on 25 hand-crafted numerical features extracted from the parsed solver metrics. Count-valued features (node counts, LP iterations, cut counts, problem dimensions) are log-transformed via $\log(1+x)$ to handle the heavy-tailed distributions observed in the dataset (node counts span six orders of magnitude), and all features are then standardized via z-score normalization fit on the training split. The network has two hidden layers of 64 units with ReLU activations and dropout (rate 0.3), and a five-output sigmoid head producing one probability per bottleneck label. The total parameter count is 6,149. Training uses binary cross-entropy loss, the AdamW optimizer with weight decay 10^{-4}, and a `WeightedRandomSampler` that oversamples training examples carrying rare positive labels (per-sample weight is the average inverse class frequency of its positive labels) to address multi-label class imbalance.

This architecture was chosen over a transformer encoder over the raw log text for two reasons: the parsed structured metrics already capture the signals the heuristic rules use, and the dataset size (141 training examples) is well below the threshold at which fine-tuning a transformer encoder becomes practical. Architectures designed for long sequences such as Longformer [6] or Big Bird [7] remain candidates for future work that ingests the raw log text directly.

4.4 Generative Explanation Model

To produce human-readable diagnostics, a sequence-to-sequence generative model (e.g., T5 or FLAN-T5) generates explanations conditioned on the solver log and predicted diagnostic labels.

Following standard autoregressive sequence modeling used in transformer architectures ([8]), the probability of the generated explanation is factorized into the product of conditional token probabilities:

$$P(\text{explanation} \mid \text{log_text}, \text{labels}) = \prod_{t=1}^{T} P(w_t \mid w_{<t}, \text{log_text}, \text{labels}) \tag{2}$$

where w_t denotes the t-th token of the generated explanation. The model is trained to produce concise summaries describing solver behavior and recommend potential improvements to model formulation or solver configuration.

4.5 LLM Approach Comparison

Three LLM-based approaches were considered for the diagnostic-generation component:

1. Prompt Engineering: Directly feeding solver logs to a pre-trained LLM with domain-specific prompts. This is the simplest approach and requires no training data.

2. Retrieval-Augmented Generation (RAG): Augmenting the LLM with a database of known solver behavior patterns and diagnostic examples. The system retrieves relevant examples based on log similarity before generating a response.

3. Fine-Tuning: Training a smaller model on the labeled dataset of solver logs and expert annotations. This approach requires the most data but may produce the most consistent domain-specific outputs.

Based on findings from the literature [1], RAG offers the best balance of accuracy and practicality for this domain given the relatively small dataset size and the need to reduce hallucination risk. The current implementation realizes the RAG approach with three coordinated design choices that emerged from iteration:

1. A 12-document curated knowledge base describing each bottleneck pattern (plus a `healthy_solve` negative class) is embedded with the `all-MiniLM-L6-v2` sentence transformer.

2. Knowledge-base retrieval is conditioned on the heuristic-rule labels: the KB is filtered to entries whose topic matches the detected labels (or `healthy_solve` when no labels fire), and the surviving entries are ranked by cosine similarity against the structured run summary. Without this conditioning the un-restricted top-k retriever surfaces bottleneck-pattern entries even on healthy runs, which biases the generator toward producing a bottleneck diagnosis where none is warranted.

3. The retrieved context, the detected labels, and a one-line narrative summary of the run are passed to `meta-llama/Llama-3.2-1B-Instruct` via its chat template. The system instruction explicitly allows "no bottleneck" as a valid output so healthy runs are not forced into a bottleneck diagnosis.

Comparison against prompting-only and fine-tuned baselines is left to future work.

5 Experimental Methodology

The experimental evaluation assesses the system along three dimensions corresponding to the research hypotheses.

5.1 Evaluation Metrics

- Bottleneck detection accuracy (H2): per-label precision, recall, F1, ROC-AUC, and average precision on a held-out validation split, plus the same metrics on the 30-run hand-reviewed gold subset described in Section 4.2.

- Recommendation quality (H3): a paired comparison of solve time and node count between the two solver configurations on the 90 MIPLIB problems run under both, used as a proxy for the kind of measurable performance shift a configuration recommendation should produce.

- Diagnostic-generation quality (H1): qualitative inspection of LLM-generated diagnostics on three worked examples spanning the bottleneck space (an excessive-branching case, a weak-root-LP case, and a healthy negative-control case), together with a check that the retrieval step surfaces the topically-correct knowledge-base entries.

5.2 Experimental Design

The 188-run dataset was split 75/25 into 141 training and 47 validation runs using `sklearn.model_selection.trai` with a fixed seed. The 30-run hand-reviewed gold subset was held out from this split for two separate purposes: characterizing the noise floor of the heuristic rules used as weak supervision (heuristic-vs-gold comparison), and measuring whether the trained classifier generalizes beyond the rules it was trained on (classifier-vs-gold comparison). The classifier was trained for 300 epochs with batch size 16 on the training split. The RAG diagnostic generator was applied to three representative runs without further training. Configuration comparisons were computed over the 87 paired MIPLIB problems for which both runs reported solve times.

6 Results

6.1 Heuristic Label Distribution

The heuristic rules described in Section 4.2, applied to the full 188-run dataset, produced the label prevalences shown in Table 1. Sixty-four percent of runs carried at least one bottleneck label and 25% carried two or more, with a mean of 0.95 labels per run.

Label	**Prevalence**
`weak_root_lp`	13.8% (26/188)
`excessive_branching`	29.3% (55/188)
`ineffective_cuts`	22.9% (43/188)
`degeneracy`	8.5% (16/188)
`presolve_weakness`	20.2% (38/188)

Table 1: Heuristic-rule label prevalence across the 188-run dataset.

6.2 Bottleneck Classification on the Validation Split (H2)

The PyTorch classifier was trained for 300 epochs, reaching a final training loss of 0.0388 and a validation loss of 0.3102. Per-label performance on the 47-run validation split is shown in Table 2. The macro-averaged F1 was 0.874 and the micro-averaged F1 was 0.860, both above the 0.85 target identified in hypothesis H2. Subset accuracy (the fraction of validation runs with all five labels predicted exactly correctly) was 0.766.

The validation-set per-label support is small (between 4 and 13 positives per label), so the per-label F1 numbers carry meaningful confidence intervals. The gold-subset comparison in the next subsection provides a complementary view with somewhat larger per-label support and an independent ground truth.

Label	Pos.	Precision	Recall	F1	ROC-AUC	AvgPrec
weak_root_lp	4	1.000	1.000	1.000	1.000	1.000
excessive_branching	13	0.857	0.923	0.889	0.977	0.946
ineffective_cuts	13	0.846	0.846	0.846	0.943	0.899
degeneracy	7	1.000	0.714	0.833	0.836	0.777
presolve_weakness	10	0.800	0.800	0.800	0.943	0.834
Macro-avg				**0.874**		

Table 2: Validation-set per-label classification metrics ($n = 47$). "Pos." is the number of positive examples in the validation split; ROC-AUC and average precision are computed on the predicted probability scores.

6.3 Heuristic and Classifier Performance vs. the Gold Subset

The 30-run hand-reviewed gold subset was used to measure (i) the supervision noise floor of the heuristic rules and (ii) whether the trained classifier generalizes beyond the rules. Table 3 reports per-label F1 for both, plus the difference.

Label	Pos.	Heuristic F1	Classifier F1	Δ
weak_root_lp	10	1.000	1.000	+0.000
excessive_branching	11	1.000	0.909	−0.091
ineffective_cuts	9	1.000	0.941	−0.059
degeneracy	5	0.833	1.000	+0.167
presolve_weakness	4	0.667	0.667	+0.000
Macro-avg		**0.900**	**0.903**	+0.003

Table 3: Per-label F1 of the heuristic rules and the trained classifier against the 30-run hand-reviewed gold subset. The classifier improves on degeneracy by suppressing the heuristic's small-problem false positives.

The macro-averaged F1 of the trained classifier (0.903) is essentially tied with that of the heuristic rules (0.900), but the per-label decomposition is informative. On the three labels where the heuristic was already perfect (weak_root_lp, excessive_branching, ineffective_cuts), the classifier reproduced or slightly underperformed the rule. On degeneracy, the classifier improved over the heuristic by 0.167 in F1 – it learned to suppress the small-problem false positives that the rule produces. On presolve_weakness, neither the rule nor the classifier resolved the small-problem false positives, suggesting that the relevant size signal is not strongly enough captured in the current feature set for the classifier to discover the correction.

6.4 H3 Recommendation Experiment

To test whether the system's diagnostic output correlates with measurable performance shifts, four configuration-picker strategies were evaluated on the 87 MIPLIB problems for which both branching_heavy and heuristic_heavy reported solve times. The ground-truth label for each problem is the configuration with the lower metric_timeline_total_seconds; heuristic_heavy was faster on 53 problems (60.9%) and branching_heavy on 34 (39.1%), with mean and median solve-time deltas of −17.4 s and −0.04 s respectively (heuristic_heavy minus branching_heavy).

A paired Wilcoxon signed-rank test [9] – chosen over the paired t-test because the per-problem deltas are heavy-tailed and not approximately normal (mean and median differ by a factor of more than 400) – fails to reject the null hypothesis of equal solve-time distributions ($W = 1728.0$, $p = 0.43$). The configurations differ on individual problems but the differences do not aggregate to a significant effect across the dataset.

Table 4 groups paired problems by which bottleneck labels fired on the `branching_heavy` log and reports how often each configuration was faster within that group. One label is sharply discriminative: every problem flagged for `degeneracy` (n=7) was faster under `heuristic_heavy`. The other four labels and the no-label group all sit close to the global 60.9% baseline.

Label	**n**	**HH wins**	**HH %**	**Mean delta (s)**
`weak_root_lp`	10	6	60.0%	−113.2
`excessive_branching`	27	14	51.9%	−50.0
`ineffective_cuts`	22	12	54.5%	−13.9
`degeneracy`	7	7	100.0%	−37.0
`presolve_weakness`	19	12	63.2%	−10.9
(no labels)	31	20	64.5%	+3.5

Table 4: Per-bottleneck configuration preference on the 87 paired MIPLIB problems. "HH wins" is the number of problems where `heuristic_heavy` was faster within each labeled subset; positive mean deltas indicate `branching_heavy` faster. Only `degeneracy` is sharply discriminative.

Table 5 compares four configuration-picker strategies against the ground truth: a deterministic alternating predictor (a stand-in for random), a majority-class predictor (always `heuristic_heavy`), a rule-based predictor that picks `heuristic_heavy` when any of `weak_root_lp`, `excessive_branching`, or `ineffective_cuts` fires on the input log, and an LLM-based predictor that asks `Llama-3.2-1B-Instruct` to choose given a non-time-leaking structural summary plus the detected labels.

Predictor	**Accuracy**	**Correct / 87**
Random (alternating)*	64.4%	56
Majority (always `heuristic_heavy`)	60.9%	53
Rule-based (label-driven)	43.7%	38
LLM-based (`Llama-3.2-1B-Instruct`)	39.1%	34

Table 5: Configuration-picker accuracy on the 87 paired problems. *The deterministic alternating predictor's 64.4% reflects a fortuitous alignment of its output sequence with the data ordering rather than a real signal; the expected accuracy of a properly random predictor is 50%.

Two findings undercut H3 in this dataset. First, the Wilcoxon test indicates that the two configurations have statistically indistinguishable solve-time distributions; the 60.9% / 39.1% split observed across the 87 problems is consistent with sampling noise around an effect size of zero, leaving essentially no signal for any picker to recover. Second, the LLM-based picker degenerated to predicting `branching_heavy` on all 87 problems – a known failure mode for small instruction-tuned models when one of two labeled options appears first in the prompt. Its 39.1% accuracy is exactly $1 - 60.9\%$, the inverse of the majority class. The rule-based picker performs marginally better at 43.7% but still falls below majority. The bottleneck taxonomy used in this work was designed for diagnosing what is wrong with a run, and this experiment indicates it does not transfer cleanly to the related but distinct task of selecting which of two solver builds is faster on a problem.

The single exception is the degeneracy label, which is a perfect predictor of heuristic_heavy preference within its 7-problem subset; the sample is too small to support a general claim but suggests that label-conditioned config selection might work for sufficiently sharp signals.

6.5 RAG-Based Diagnostic Generation (H1)

After conditioning retrieval on the heuristic-rule labels, the retrieval component surfaced topically-aligned knowledge-base entries on all three worked examples. For the multi-label run flagged as excessive_branching and ineffective_cuts (branching_heavy::unitcal_7), the top retrievals were two excessive_branching entries plus one ineffective_cuts entry. For the weak_root_lp run (branching_heavy::gfd-schedulen180f7d50m30k18), both retrievals were weak_root_lp entries. For the healthy air05 run, both retrievals were healthy_solve entries.

The generation step, using meta-llama/Llama-3.2-1B-Instruct, produced coherent single-paragraph diagnostics on all three examples. On the unitcal_7 run the model identified the dominant bottleneck with concrete numbers from the input summary ("a high node count (2,669,926) relative to the number of variables (25,755)") and recommended specific configuration changes drawn from the retrieved context: strong branching, pseudocost branching, symmetry-breaking constraints, and increased cut aggressiveness. It also explicitly noted the secondary ineffective_cuts factor and connected it to the primary diagnosis. On the gfd-schedulen180f7d50m30k18 run the model correctly identified the weak-root-LP bottleneck, referenced the no-solution termination, and recommended increased presolve passes and more aggressive cut generation. On the healthy air05 run the model opened with "appears to be healthy, with no detected bottlenecks" but introduced a fabricated quantitative claim ("the presolve variable reduction was only 15%, which is below the 10% threshold typically required for a healthy solver run") that is not present in the knowledge base or in the heuristic rules. The qualitative diagnosis was correct but the supporting rationale on the healthy run included a hallucination.

An earlier configuration of the system using FLAN-T5-base with a longer multi-section prompt produced degenerate output on all three examples (token-repetition loops or fragmentary single-line responses). The transition from FLAN-T5-base to Llama-3.2-1B-Instruct, combined with a shortened chat-formatted prompt, label-conditioned retrieval, and an explicit "no bottleneck" allowance in the system instruction, was responsible for the shift to coherent diagnostics. The remaining hallucination on the healthy-run case suggests that even a stronger instruction-tuned model produces unsupported claims when asked to write a paragraph about a run with no actionable issues; mitigations are discussed in Section 7.

To complement the qualitative discussion above with a quantitative measure, the three worked diagnostics were scored on a structured five-criterion 1–5 Likert rubric (factual accuracy, completeness, actionability, terminology correctness, conciseness; rubric definitions and per-score anchors documented in the accompanying diagnostic_rubric.xlsx). Per-diagnostic scores are shown in Table 6.

Three patterns emerge. First, the two bottleneck-case diagnostics (unitcal_7 and gfd-schedulen180...) score in the high teens to low twenties out of 25 – usable for a practitioner but not flawless, with deductions concentrated on factual accuracy and actionability. The gfd-schedulen180 factual-accuracy score of 2 in particular reflects two issues: the diagnostic asserts "the LP bound being far from the integer optimum" without evidence in the run summary to support that specific causal claim, and the recommended parameters (cut_aggressiveness=2 or 3, presolve passes to 5) are borrowed from generic-solver knowledge-base text and do not match Rose's actual control vocabulary. Second, the healthy-run diagnostic scores the lowest at 10/25, with floor scores on factual accuracy and actionability driven by the fabricated 10% presolve threshold and the recommenda-

Diagnostic	FA	COMP	ACT	TERM	CONC	Total
`unitcal_7`	4	5	4	5	3	21/25
`gfd-schedulen180...`	2	5	3	4	4	18/25
`air05` (healthy)	1	3	1	2	3	10/25
Per-criterion mean	2.33	4.33	2.67	3.67	3.33	**3.27**/5

Table 6: Diagnostic-quality rubric scores across the three worked examples. FA = factual accuracy; COMP = completeness; ACT = actionability; TERM = terminology correctness; CONC = conciseness. Each criterion uses a 1–5 Likert scale with shared anchors (5 = excellent, 3 = acceptable, 1 = unacceptable). Scoring was performed by a single expert reviewer (the project author).

tion to "increase the presolve reduction threshold to 20% or more," which references a parameter that does not exist (presolve reduction is an output metric, not a configurable threshold). Third, the per-criterion decomposition shows that completeness (4.33) and terminology correctness (3.67) are the strongest dimensions – the system reliably names the right bottlenecks and uses domain language correctly – while factual accuracy (2.33) and actionability (2.67) are the weakest. This is the H1 caveat already discussed, now quantified: the diagnostic generator is substantially stronger at *identifying* bottlenecks than at *justifying* them or at *recommending* solver-specific actions.

7 Conclusion and Future Work

This project explored the use of large language models to interpret optimization solver logs and provide automated diagnostics for mathematical optimization problems. The Optimization Log Copilot pipeline – structured log parsing, heuristic-rule weak supervision, a multi-label PyTorch classifier, and a retrieval-augmented diagnostic generator – was implemented end-to-end and evaluated against an independent 30-run hand-reviewed gold subset. The following findings address the three research hypotheses stated in Section 3.

H2 (Bottleneck Detection) is supported. The trained classifier achieved macro-F1 of 0.874 and micro-F1 of 0.860 on the held-out validation split, both above the 0.85 target identified in the proposal. Against the independent 30-run gold subset the classifier achieved macro-F1 of 0.903, essentially tied with the heuristic rules used to weakly supervise it (0.900). The classifier did not simply reproduce its supervision: on the `degeneracy` label it improved on the rules by 0.167 in F1 by learning to suppress the small-problem false positives that the rule produces, demonstrating that the model captured at least one structural pattern that the heuristic missed.

H1 (Diagnostic Summarization) is supported with one caveat. After conditioning knowledge-base retrieval on the heuristic-rule labels and replacing the generator with `meta-llama/Llama-3.2-1B-I` prompted via its chat template, the system produced coherent single-paragraph diagnostics on all three worked examples. Structured rubric scoring across five criteria (Section 6, Table 6) yields a mean of 3.27/5 across the 15 individual scores, with the two bottleneck-case diagnostics scoring 21/25 and 18/25 and the healthy-run diagnostic scoring 10/25. The per-criterion decomposition shows completeness (mean 4.33) and terminology correctness (mean 3.67) as the strongest dimensions, while factual accuracy (2.33) and actionability (2.67) are the weakest – driven primarily by the healthy-run case, on which the model fabricated a non-existent "10% presolve reduction threshold" and recommended adjusting a parameter that does not exist. The qualitative diagnosis was correct on every case (every bottleneck label was named, every healthy run was recognized), but the supporting rationale on the healthy run included unsupported quantitative claims. Larger or

stronger instruction-tuned generators are likely to reduce this kind of confabulation, and an explicit instruction not to invent numerical thresholds would help further.

H3 (Recommendations correlate with measurable performance shifts) is not supported. A paired Wilcoxon signed-rank test on the 87 MIPLIB problems run under both configurations failed to reject the null hypothesis of equal solve-time distributions ($p = 0.43$), indicating no aggregate signal for the recommendation task to recover at the dataset level. Both predictors evaluated – a rule-based picker driven by the heuristic bottleneck labels (43.7% accuracy) and an LLM-based picker using `Llama-3.2-1B-Instruct` (39.1% accuracy, after collapsing to a single class) – fell below the majority-class baseline of 60.9%. The single positive finding is that the `degeneracy` label was a perfect predictor of `heuristic_heavy` preference within its 7-problem subset, but the sample is too small to support a general claim. The bottleneck taxonomy used in this work was designed for diagnosing what is wrong with a run; this result indicates that it does not transfer cleanly to the task of selecting between two solver builds. A predictor designed specifically for the configuration-selection task – rather than reusing the diagnostic labels – would be a natural direction for follow-up work.

The contribution of this work is the demonstrated feasibility of the diagnostic pipeline: a working data parser that ingests arbitrary Rose solver logs, an empirically-validated set of heuristic bottleneck rules with a documented noise floor (macro-F1 = 0.900 against expert review), a learned classifier that meets the proposed accuracy target while improving on its supervision in one labeled category, and a label-conditioned RAG diagnostic generator that produces coherent qualitative diagnoses across the bottleneck space. The H3 negative result demarcates the boundary of what these diagnostic labels can do: they are useful for explaining solver behavior to a practitioner but do not, in their current form, support automated configuration selection.

7.1 Limitations

Three limitations of the current work are worth noting. First, the heuristic supervision used to train the bottleneck classifier exhibits a known precision-only noise pattern, with false positives concentrated on the `degeneracy` and `presolve_weakness` labels for instances with fewer than approximately 50 variables. The pattern is documented in Section 4.2 but deliberately not corrected against the hand-reviewed gold subset, since doing so would contaminate that subset's role as the evaluation instrument. Second, the dataset is drawn from a single solver (Rose) and two configurations; behavior of the diagnostics on logs from other solvers such as Gurobi, HiGHS, or CPLEX is not characterized. Third, the diagnostic-quality rubric is applied by a single expert reviewer; inter-annotator agreement was not measured.

7.2 Future Work

Several concrete next steps remain. Given the H3 negative result, the most direct follow-up is to design a configuration-selection predictor distinct from the diagnostic taxonomy – for example, a small classifier trained on structural problem features (size, density, constraint-type breakdown, presolve-reduction profile) with the per-problem "which configuration was faster" label as supervision. The 87 paired problems are too small to robustly train such a model, but they are sufficient to characterize whether any signal exists beyond the bottleneck labels evaluated here. A second item is extending the diagnostic-quality rubric (Section 6) from the three worked examples to the full 30-run gold subset, which would tighten the per-criterion mean estimates and characterize the rate of the fabricated-quantitative-claim failure mode at scale; a calibrated LLM-as-judge could substitute for expert review at that scale. A third item is comparing the RAG approach against

a prompting-only baseline and (data-permitting) a fine-tuned baseline, completing the three-way comparison proposed in Section 4. Beyond these, broader directions include application to logs from additional solvers such as Gurobi, HiGHS, and CPLEX; interactive solver debugging via a conversational interface; and integration into existing solver frameworks.

References

[1] S. Akhtar, S. Khan, and S. Parkinson, "LLM-based event log analysis techniques: A survey," *arXiv preprint arXiv:2502.00677*, 2025. [Online]. Available: https://arxiv.org/abs/2502.00677

[2] M. Landauer, S. Onder, F. Skopik, and M. Wurzenberger, "Deep learning for anomaly detection in log data: A survey," *Machine Learning with Applications*, vol. 12, p. 100470, 2023. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2666827023000233

[3] Y. Bengio, A. Lodi, and A. Prouvost, "Machine learning for combinatorial optimization: A methodological tour d'horizon," *arXiv preprint arXiv:1811.06128*, 2018. [Online]. Available: https://arxiv.org/abs/1811.06128

[4] J. Kotary, F. Fioretto, P. Van Hentenryck, and B. Wilder, "End-to-end constrained optimization learning: A survey," in *Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI)*, 2021. [Online]. Available: https://arxiv.org/abs/2103.16378

[5] M. Yazdani, M. Mostajabdaveh, S. Aref, and Z. Zhou, "EvoCut: strengthening integer programs via evolution-guided language models," *arXiv preprint arXiv:2508.11850*, 2025. [Online]. Available: https://arxiv.org/abs/2508.11850

[6] I. Beltagy, M. E. Peters, and A. Cohan, "Longformer: The long-document transformer," *arXiv preprint arXiv:2004.05150*, 2020. [Online]. Available: https://arxiv.org/abs/2004.05150

[7] M. Zaheer, G. Guruganesh, A. Dubey, J. Ainslie, C. Alberti, S. Ontañón, P. Pham, Q. Wang, A. Ravula, L. Yang, and A. Ahmed, "Big Bird: Transformers for longer sequences," *arXiv preprint arXiv:2007.14062*, 2020. [Online]. Available: https://arxiv.org/abs/2007.14062

[8] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer," *Journal of Machine Learning Research*, vol. 21, no. 140, pp. 1–67, 2020.

[9] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945. [Online]. Available: https://doi.org/10.2307/3001968